

Chapter 5

Linux Event Driven Programming

5 Lecture · 영상 86 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L21 Blocking 과 Non-Blocking I/O

L22 select 와 poll 내부 구조

L23 epoll 내부 구조

L24 Level Trigger 와 Edge Trigger

L25 eventfd / timerfd / signalfd / inotify

L21

Blocking 과 Non-Blocking I/O

Chapter 5. Linux Event Driven Programming

read() 는 Data 가 올 때까지 잠든다 . 입력이 둘 이상이면 문제가 된다

영상 16 분

▶ 애니메이션 [C5-1 Blocking vs Non-Blocking](#)

이 Chapter 의 범위

Network Programming 은 다루지 않는다 . select / poll / epoll 은 Network API 가 아니라 **fd** 의 상태 변화를 기다리는 범용 장치

pipe eventfd timerfd signalfd inotify

L21	Blocking 의 문제	L24	Level / Edge Trigger
L22	select, poll	L25	Timer, Signal, Thread 통지 , File 변화를 fd 로
L23	epoll		

문제 상황

```
$ ./two_pipes  
[ 3.0s] slow: slow#1  
[ 3.0s] fast: fast#1 fast#2 fast#3 ... fast#10
```

fast 는 0.2 초마다 도착하고 있었지만 **3 초 동안 하나도 처리되지 못했다**

→ *Terminal 전환*

Blocking: 잠든다는 것의 실체

```
Reader: read() → Buffer 가 비었다 → Wait Queue 에 자신을 등록 → schedule()  
Writer: write() → Buffer 에 Data → wake_up(Wait Queue) → Reader 가 Runnable
```

PID	STAT	WCHAN	COMMAND
2412	S	pipe_read	block_read

- CPU 를 쓰지 않는다 . 코드가 단순하다
 - L19 의 futex 와 같은 Pattern. Kernel 의 모든 Blocking 이 이 구조
- file_operations 의 **.poll** = " 준비됐는가 ? 아니면 어느 Wait Queue 에 걸어야 하는가 ?"

O_NONBLOCK과 EAGAIN

```
flags = fcntl(fd, F_GETFL);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

read()	Blocking	Non-blocking
Data 있음	n > 0	n > 0
없음, Writer 살아 있음	잠든다	-1, EAGAIN
없음, Writer 가 닫음	0 (EOF)	0 (EOF)

EAGAIN 은 오류가 아니다 · 일반 File 에는 효과가 없다

Busy Polling 의 비용

read() 를 12,574,142 번 호출한 끝에 5 Byte 를 읽었습니다
그동안 사용한 CPU 시간 : user 0.72 초 + system 1.27 초

← 기다린 2 초 내내 CPU 100%

usleep 을 넣으면 ? → CPU 는 줄지만 반응이 그만큼 늦어진다

Non-blocking 은 " 언제 다시 시도하면 되는지 알려 주는 장치 " 와 짝이 되어야 한다

→ Terminal 전환

쓰기에도 **Blocking** 이 있다

```
write 가 멈춘 시점 : 누적 65536 Byte, errno = 11 (EAGAIN)  
F_GETPIPE_SZ = 65536
```

Pipe Buffer 64KB. 가득 차면 Blocking write 는 잠든다

→ 읽기 가능 (EPOLLIN) 뿐 아니라 쓰기 가능 (EPOLLOUT) 도 기다릴 Event

Blocking vs Non-Blocking

화면 전환 → [animation.html](#)

Blocking read → Busy Polling → 입력이 들일 때

▶ 애니메이션 [C5-1 Blocking vs Non-Blocking](#)

해법의 후보들

방법	문제
fd 마다 Thread	수천 개면 Memory, Context Switch, Contention (L20)
Non-blocking + Busy Polling	CPU 100%
SIGIO	Handler 의 제약 . 거의 안 씀
I/O Multiplexing	" 이 중 하나라도 준비되면 깨워 줘 " → select, poll, epoll
비동기 I/O	Chapter 6 io_uring

핵심 : 기다리는 곳을 한 군데로 모은다

정리

- Blocking = Wait Queue 에서 잠든다 . 한 번에 하나만
O_NONBLOCK = EAGAIN 으로 즉시 return. 혼자서는 Busy Polling
- 쓰기도 Block 된다
- I/O Multiplexing = 여러 Wait Queue 에 동시에 걸기

다음 : L22. 1983 년의 select

L22

select 와 poll 내부 구조

호출할 때마다 목록 전체를 넘기고 , Kernel 은 전부 검사한다

영상 17 분

▶ 애니메이션 [C5-2 select / poll 동작](#)

문제 상황

N =	10	poll 1 회	평균	1.0 us
N =	100	poll 1 회	평균	4.8 us
N =	1000	poll 1 회	평균	43.7 us
N =	10000	poll 1 회	평균	687.1 us

활동하는 fd 는 언제나 **1** 개인데 N 에 비례해 느려진다

select 로 두 입력을 동시에

```
while (open_cnt > 0) {  
    FD_ZERO(&rset);  FD_SET(...);           /* 1. 매번 다시 만든다 */  
    select(maxfd + 1, &rset, NULL, NULL, NULL); /* 2. 전체를 넘긴다 */  
    for (...) if (FD_ISSET(fds[i], &rset)) { read ... } /* 3. 전부 검사한다 */  
}
```

[0.2s] fast#1 [0.7s] fast#2 [1.2s] slow#1 [1.2s] fast#3 ...

→ *Terminal 전환*

select 의 불편 → poll

	select	poll
자료 구조	Bitmask 3 개 , 고정 1024bit	struct pollfd 배열
fd 번호 제한	1024 미만	없음
입력 / 출력	같은 변수 (덮어쓰기)	events / revents 분리
추가 정보		POLLHUP, POLLERR, POLLNVAL

`nfds` = 개수가 아니라 **최대 fd 번호 + 1**

→ Terminal 전환: `./poll_demo`

Kernel 안에서 일어나는 일

```
poll(pfds, N, timeout)
```

1. 배열 전체를 User → Kernel 복사
2. N 개를 순회 : f_op->poll() - "준비됐나?" + 그 fd 의 Wait Queue 에 나를 등록
3. 준비된 것이 없으면 Sleep
4. 누군가 깨우면 (누구인지 모른다) → 2 번을 처음부터 다시
5. N 개의 Wait Queue 에서 나를 제거
6. 배열 전체를 Kernel → User 복사
7. (User) N 개를 순회하며 revents 검사

Kernel 은 호출 사이에 아무것도 기억하지 않는다

select / poll 동작

화면 전환 → [animation.html](#)

select → poll → $N = 10,000$, Ready = 1

▶ 애니메이션 [C5-2 select / poll 동작](#)

FD_SETSIZE 의 벽

```
FD_SETSIZE = 1024, sizeof(fd_set) = 128 Byte
```

```
읽기용 fd = 2000
```

```
select: FD_SET(2000, &set) → 범위 밖 Memory 를 건드린다 (Buffer Overflow)
```

```
poll : poll() = 1, revents = 0x1
```

제한은 fd 의 개수가 아니라 번호 . `ulimit -n` 을 올린 뒤에 나타나는 Bug

새 코드에서는 **select** 를 쓰지 않는다

→ *Terminal 전환*

N 이 커지면 : C10K

$N = 10,000$, Event 초당 10,000 번 $\rightarrow 0.69\text{ms} \times 10,000 = \mathbf{6.9 \text{ 초 / 초}}$

- 연결 대부분이 Idle 인 Server 에서 무너진다

비용이 **활동하는 fd** 수가 아니라 **감시하는 fd** 수에 비례한다는 것이 본질

$\rightarrow$ Terminal 전환: `./scale_poll`

그래도 쓰는 곳

poll	fd 수십 개 이하 , 이식성 (POSIX), 단순한 코드
select	오래된 코드 , Windows 호환 , μ s sleep 관용구
그 외	Linux에서는 epoll

정리

- 하나의 Thread 가 여러 Wait Queue 에 동시에 걸려 기다린다
- select: 1024bit Bitmask, 덮어쓰기 / poll: 배열, 제한 없음

Kernel 이 Stateless → 호출마다 복사 + 전체 검사 + 등록 / 해제 → **$O(N)$**

다음 : L23. Kernel 이 기억하게 하자

L23

epoll 내부 구조

등록은 한 번 , 기다릴 때는 준비된 것만 돌려받는다

영상 18 분

▶ **애니메이션** [C5-3 epoll: Interest List와 Ready List](#)

문제 상황과 결과

N	poll (us)	epoll (us)	poll/epoll
10	0.9	0.5	2x
100	4.6	0.5	10x
1000	47.4	0.5	103x
10000	751.0	0.5	1602x

epoll 은 N 과 무관하게 $0.5\mu\text{s}$

세 개의 System Call

```
epfd = epoll_create1(EPOOL_CLOEXEC);  
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);  
n = epoll_wait(epfd, events, MAX, -1);  
for (i = 0; i < n; i++) handle(events[i]);
```

```
/* Instance 생성 . 이것도 fd */  
/* 등록 : 한 번만 */  
/* 인자에 fd 목록이 없다 */  
/* 준비된 것만 들어 있다 */
```

poll 이 섞어 놓았던 " 알려 주기 " 와 " 기다리기 " 를 분리

→ Code + Terminal: *epoll_demo*

Interest List 와 Ready List

```
epfd → struct eventpoll
    └─ Interest List (Red-Black Tree)    ← epoll_ctl 이 수정
    └─ Ready List    (Linked List)      ← Callback 이 추가
    └─ epoll_wait 로 잠든 Thread 의 Wait Queue
```

epoll_ctl(ADD): epitem 생성 → Tree 에 삽입 → 그 fd 의 Wait Queue 에 **Callback** 을 한 번 걸어 둔다

Event 발생 : wake_up → ep_poll_callback → Ready List 에 추가 (O(1)) → 잠든 Thread 를 깨움

epoll_wait: Ready List 만 본다 → O(준비된 수)

poll 과의 비교

	poll	epoll
관심 목록	User 배열 . 매번 복사	Kernel 안
Wait Queue 등록	호출마다 N 개 등록 / 해제	ADD 때 한 번
누가 준비됐는지	깨어나서 N 개 재검사	Event 를 일으킨 쪽이 Callback 으로 직접 추가
결과	N 개 배열에 표시	준비된 것만
비용	$O(N)$	$O(\text{준비된 수})$

" 전부에게 물어보기 " → " 일이 생기면 알려 주기 "

epoll: Interest List 와 Ready List

화면 전환 → [animation.html](#)

FD 수와 Ready 수를 바꿔 가며

▶ 애니메이션 [C5-3 epoll: Interest List와 Ready List](#)

"epoll 이 무조건 빠르다 " 가 아니다

- $N = 10$ 에서는 2 배

fd 가 몇 개뿐이거나 거의 모든 fd 가 항상 활동 중이면 차이가 없거나 epoll_ctl 만큼 손해
압도적인 것은 많은 fd 중 일부만 활동하는 경우 = 대부분의 Server

→ Terminal 전환: `./scale_compare`

epoll_event.data

```
typedef union epoll_data { void *ptr; int fd; uint32_t u32; uint64_t u64; } epoll_data_t;
```

Kernel 이 해석하지 않고 그대로 돌려주는 **8Byte**. union 이므로 하나만

data.fd	예제용
data.u32	fd 의 종류 → switch (L25)
data.ptr	연결별 구조체 포인터 . 실전의 표준 . fd → 객체 검색이 필요 없다

들여다보기와 주의점

```
$ cat /proc/PID/fdinfo/<epfd>
tfd:  5 events: 80000019 ...    ← EPOLLET | IN | ERR | HUP
tfd:  6 events: 40000019 ...    ← EPOLLONESHOT
tfd:  3 events:          19 ...
```

일반 File 은 등록 불가 (EPERM)

epoll 은 fd 번호가 아니라 **struct file** 을 감시 → dup/fork 가 있으면 close 해도 남는다 →

close 전에 EPOLL_CTL_DEL

EINTR 처리 , Multi-Thread 에서는 EPOLLONESHOT, Linux 전용

→ Terminal 전환: `./epoll_inspect`

정리

- "알려 주기"와 "기다리기"의 분리. 목록은 Kernel 안에
- Interest List(Tree) + Callback + Ready List(List)
- $O(\text{준비된 수})$. $N = 10,000$ 에서 poll의 1,600배
- data.ptr
- 일반 File 불가, close 전 DEL

다음 : L24. EPOLLET

L24

Level Trigger 와 Edge Trigger

" 있는 동안 계속 " vs " 생긴 순간 한 번 "

영상 16 분

▶ **애니메이션** [C5-4 Level Trigger vs Edge Trigger](#)

문제 상황

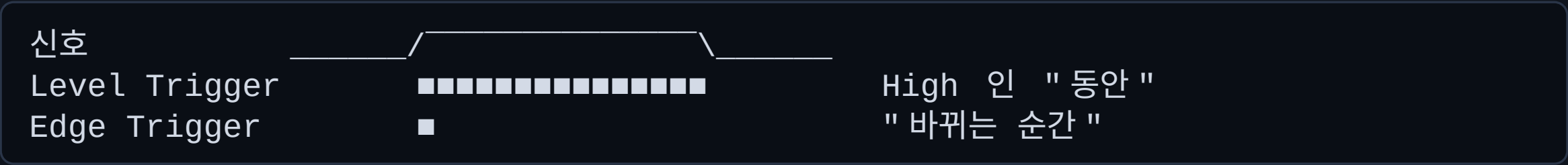
Pipe 에 "HELLO LINUX" 11Byte. Event 마다 **3Byte** 만 읽는다

```
LT:  #1 "HEL"  #2 "LO "  #3 "LIN"  #4 "UX"  #5 Timeout ( 남은 Data 0)
ET:  #1 "HEL"  #2 Timeout ( 남은 Data 8 Byte)  #3 Timeout  #4 Timeout ...
```

ET 에서 "LO LINUX" 는 영원히 읽히지 않는다

→ *Terminal* 전환

용어의 유래



	질문
LT (기본)	" 지금 읽을 수 있는가 ?"
ET (EPOLLET)	" 새로운 일이 생겼는가 ?"

select, poll 은 LT 만 . ET 는 **Kernel** 이 상태를 기억하는 **epoll** 이어서 가능

Kernel 에서의 차이

```
epoll_wait: Ready List 에서 epitem 을 꺼낸다 → f_op->poll 로 상태 확인 → events[] 에 복사
├─ LT: 그 epitem 을 Ready List 에 다시 넣는다 → 다음에도 확인받는다 . 비면 그때 버려진다
└─ ET: 다시 넣지 않는다 → 다음에 들어오는 것은 Callback 이 불릴 때뿐
                        = Writer 가 wake_up 할 때 = 새 Data 도착
```

Kernel 이 잊은 것이 아니라 다시 알려 줄 계기가 없는 것

LT vs ET + 직접 해 보기

화면 전환 → [animation.html](#)

LT → ET(3Byte 만) → ET(EAGAIN 까지) → 아래의 Sandbox

▶ **애니메이션** [C5-4 Level Trigger vs Edge Trigger](#)

Edge = " 새 Data 의 도착 "

1. 비어 있음	<code>epoll_wait = 0</code>	(0 Byte)	
2. write 11 Byte	<code>epoll_wait = 1</code>	(11 Byte)	
3. 3 Byte 만 읽음	<code>epoll_wait = 0</code>	(8 Byte)	
4. write 1 Byte (새 Data 도착)	<code>epoll_wait = 1</code>	(9 Byte)	← 비어 있지 않았는데도
5. 아무 일도 없음	<code>epoll_wait = 0</code>	(9 Byte)	

Data 가 계속 들어오는 동안에는 Bug 가 숨는다 . **마지막 Message** 를 덜 읽었을 때만 멈춘다

→ Terminal 전환

ET 의 올바른 Pattern

```
/* 1. fd 는 반드시 O_NONBLOCK */
for (;;) {
    n = read(fd, buf, sizeof(buf));
    if (n > 0) { process(); }
    else if (n < 0 && errno == EAGAIN) break; /* 다 읽었다 → epoll_wait 로 */
    else { /* 0: EOF, 그 외: 오류 */ break; }
}
```

O_NONBLOCK 없이 이 **Loop** 를 돌리면 마지막 **read** 에서 영원히 **Block**

Starvation 주의 : 한 번에 읽는 양에 상한

→ Code + Terminal: *et_drain*

왜 굳이 ET를 쓰는가

	LT	ET
안전성	덜 읽어도 다시 알려 준다	덜 읽으면 멈춘다
epoll_wait 횟수	조금씩 읽으면 많아진다	Event 당 한 번
Kernel 비용	되내기 + poll 재확인	없음
EPOLLOUT	걸어 두면 쉬지 않고 return → 필요할 때만 MOD로 켜고 꺼야	걸어 둔 채로 둘 수 있다
쓰는 곳	libevent, Redis, 대부분	nginx, Envoy, Go, tokio

EPOLLONESHOT, 선택 기준

EPOLLONESHOT: 한 번 통지 후 비활성화 . 처리를 마친 Thread 가 EPOLL_CTL_MOD 로 다시 켜다 → " 한 fd 는 한 번에 한 Thread 만 "

1. 기본은 LT

2. ET 는 고성능 Server, Runtime/Library

ET 를 쓴다면 O_NONBLOCK + EAGAIN 까지를 **예외 없이**

3. eventfd, timerfd, signalfd 는 LT 로 충분

정리

- LT: 준비된 동안 계속 / ET: 도착한 순간 한 번
- 차이 = `epoll_wait` 후 Ready List 에 되넣는가
- ET 에서 남긴 Data 는 다시 통지되지 않는다
- ET = `O_NONBLOCK` + `EAGAIN` 까지
- 기본은 LT

다음 : L25. Timer, Signal, Thread 통지 , File 변화도 fd 로

L25

eventfd / timerfd / signalfd / inotify

fd 가 아닌 것도 fd 로 만든다 . 그래서 하나의 epoll Loop 가 모든 Event 를 처리한다

영상 19 분

▶ 애니메이션 [C5-5 하나의 Event Loop](#)

문제 상황

epoll_wait 에서 자는 Thread 를 깨울 수 있는 것은 **fd 의 Event** 뿐

기다리는 것	전통적인 방법	문제
Thread 의 " 끝났어요 "	condition variable	cond_wait 와 epoll_wait 를 동시에 기다릴 수 없다
1 초마다	SIGALRM, Timeout 계산	Handler 제약 , 복잡
SIGINT, SIGTERM	Signal Handler	아무 때나 끼어든다 . async-signal-safe
File 변화	주기적 stat	Polling

Linux 의 답 : " 전부 **fd** 로 만들어 줄 테니 **epoll** 에 넣어라 "

eventfd: Kernel 안의 Counter 하나

```
int efd = eventfd(0, EFD_NONBLOCK | EFD_CLOEXEC);
write(efd, &one, 8);          /* Counter += 1, 기다리는 쪽을 깨운다 */
read(efd, &v, 8);             /* v = Counter, Counter = 0 */
```

eventfd 에서 3 을 (를) 읽음 → 누계 3

eventfd 에서 2 을 (를) 읽음 → 누계 5 ← 통지 5 번, read 2 번, 유실 0

Pipe 대비 : fd 1 개 , 8Byte, 밀려도 **합산**되고 넘치지 않는다

→ *Code + Terminal*

timerfd: Timer 가 fd 가 된다

```
tfd = timerfd_create(CLOCK_MONOTONIC, TFD_NONBLOCK | TFD_CLOEXEC);
timerfd_settime(tfd, 0, &its, NULL);          /* 첫 만료 0.5s, 주기 0.2s */
read(tfd, &expirations, 8);                  /* 지난번 읽은 뒤 만료된 횟수 */
```

```
[0.50s] 1   [0.70s] 1   [1.60s] 4   [1.70s] 1   [1.90s] 1
           ↑ 0.7 초 늦게 읽음 : 놓친 횟수를 알려 준다 . 이후 주기는 밀리지 않는다
```

Process 당 개수 제한 없음 · 주기 작업에는 CLOCK_MONOTONIC

→ *Terminal 전환*

signalfd: Signal 을 read 로 받는다

```
sigprocmask(SIG_BLOCK, &mask, NULL);           /* 1. 먼저 Block */
sfd = signalfd(-1, &mask, SFD_NONBLOCK | SFD_CLOEXEC);
read(sfd, &si, sizeof(si));                     /* Signal 하나 = 구조체 하나 */
```

Signal 10 (User defined signal 1), 보낸 pid 2590
→ 설정을 다시 읽습니다 (printf, malloc 마음대로)
Signal 15 (Terminated) → Graceful Shutdown

끼어들기가 없다 → Race 없음, 함수 제약 없음, 보낸 pid 도 안다

→ Terminal 전환

signalfd 의 주의점

먼저 Block. 안 하면 기본 동작 (종료) 이나 Handler 로 간다

Multi-Thread: **Thread** 를 만들기 전에 pthread_sigmask → 모든 Thread 가 상속

SIGKILL, SIGSTOP 불가 · SIGSEGV, SIGBUS 는 부적합

fork + exec: Mask 가 유지된다 . Child 에서 exec 전에 복원

inotify: File System 의 변화

```
$ touch watched/a; echo hi >> watched/a; mv watched/a watched/b; rm watched/b  
CREATE a / CLOSE_WRITE a / MODIFY a / CLOSE_WRITE a / MOVED_FROM a / MOVED_TO b / DELETE b
```

IN_CLOSE_WRITE 를 기다린다 (IN_MODIFY 는 write 마다)

rename 으로 교체되는 File(L15 의 안전한 저장) 은 inode 가 바뀐다 → **Directory** 를 **Watch**

재귀 아님 , max_user_watches, IN_Q_OVERFLOW, NFS 미지원

→ *Terminal 전환 (Terminal 두 개)*

공통점 : anon_inode + file_operations

3 -> anon_inode:[signalfd] 4 -> anon_inode:[eventpoll]

fd	.read	.poll 이 EPOLLIN 인 조건
eventfd	Counter	Counter > 0
timerfd	만료 횟수	> 0
signalfd	signalfd_siginfo	Pending Signal
inotify	inotify_event 들	Queue 가 비지 않음
epoll	-	Ready List 가 비지 않음

.poll 만 구현하면 무엇이든 **epoll** 의 대상 → pidfd, userfaultfd ...

하나의 Event Loop

화면 전환 → animation.html

Timer → Worker(eventfd) → File 변화 → SIGTERM(Graceful Shutdown)

▶ 애니메이션 [C5-5 하나의 Event Loop](#)

전부 합치기 : event_loop.c

```
while (running) {
    n = epoll_wait(epfd, events, 8, -1);          /* 기다리는 곳은 여기 하나 */
    for (i = 0; i < n; i++)
        switch (events[i].data.u32) {
            case SRC_TIMER: ... case SRC_WORKER: ... case SRC_INOTIFY: ...
            case SRC_SIGNAL: running = 0; break;
        }
}
/* 정리 : Worker join, close */
```

Handler 가 한 Thread 에서 순서대로 → **Lock** 이 필요 없다

Handler 하나가 오래 걸리면 전체가 멈춘다 → **Worker + eventfd**

nginx, Redis, Node.js, systemd 의 구조 = **Final Project** 의 뼈대

→ *Code + Terminal*

Chapter 5 정리

Blocking = Wait Queue. 한 번에 하나 L21
select / poll = 매번 전부 넘기고 전부 검사. O(감시하는 수) L22
epoll = Interest List + Callback + Ready List. O(준비된 수) L23
LT: 준비된 동안 / ET: 도착한 순간 → NONBLOCK + EAGAIN L24
eventfd / timerfd / signalfd / inotify → 하나의 Event Loop L25

Worksheet 20 문제 → Chapter 6: io_uring (" 일반 File 은 epoll 에 넣을 수 없다 ")

다음 시간 · Chapter 6

Modern Asynchronous I/O — io_uring

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog